

SystemVerilog Assignment 2: Arrays & Procedural Routines

Ziad Mostafa Abdelaziz

July 16, 2026

1 Introduction

This report documents the verification of the `multi_op_processor` combinational DUT using SystemVerilog's verification constructs, specifically dynamic arrays, queues, and associative arrays.

2 Technical Choices of Data Types

As requested, the testbench employs specific SystemVerilog data structures to manage the stimulus and response collection:

- **Dynamic Array (`stim_t stim_arr[]`):** We chose a dynamic array to hold the generated stimulus packets. Since the simulation requires re-configuring the database size from 100 locations (Pass 1) to 200 locations (Pass 2 and Pass 3), a dynamic array is ideal because its sizing can be altered dynamically at run-time using the `new[size]` operator. Each element is a structured packet `stim_t` containing the 8-bit `data_in` and 2-bit `op_sel`.
- **Queue (`bit [7:0] actual_q[$]`):** A queue was selected to collect the simulated output data values from the DUT. Queues are ideal for collecting output streams chronologically. Using `push_back()`, we can dynamically accumulate outputs without needing to predetermine the array size, matching the stream-like output capture.
- **Associative Array (`bit [7:0] expected_aa[int]`):** An associative array, indexed by an integer (representing the stimulus index), is used to store the golden model values. Associative arrays represent sparse or mapped memory structures perfectly. It associates each execution test index directly to the target expected output value.

3 Code Listings

3.1 Design Module: `multi_op_processor.sv`

Listing 1: Design under test implementation

```
1 module multi_op_processor (  
2     input  [7:0] data_in,  
3     input  [1:0] op_sel,  
4     output reg [7:0] data_out  
5 );  
6  
7     always @(*) begin
```

```

8         case (op_sel)
9             2'b00: data_out = data_in + 1;          // Increment
10            2'b01: data_out = data_in - 1;          // Decrement
11            2'b10: data_out = ~data_in;             // Bitwise NOT
12            2'b11: data_out = data_in << 1;         // Left shift by 1
13            default: data_out = data_in;
14        endcase
15    end
16
17 endmodule

```

3.2 Testbench Module: tb_multi_op_processor.sv

Listing 2: SystemVerilog testbench utilizing routines and arrays

```

1 `timescale 1ns/1ps
2
3 module multi_op_processor_tb;
4
5     typedef struct packed {
6         bit [7:0] data_in;
7         bit [1:0] op_sel;
8     } stim_t;
9
10    stim_t          stim_arr[];          // dynamic array
11    bit [7:0]        actual_q[$];        // queue
12    bit [7:0]        expected_aa[int];    // associative array
13
14    logic [7:0] data_in;
15    logic [1:0] op_sel;
16    logic [7:0] data_out;
17
18    int num_checks;
19    int num_errors;
20
21    multi_op_processor dut (
22        .data_in (data_in),
23        .op_sel (op_sel),
24        .data_out (data_out)
25    );
26
27    function void configure_stim_storage(int size);
28        stim_arr = new[size];
29        actual_q.delete();
30        expected_aa.delete();
31        $display("[CONFIG] Configured for %0d entries", size);
32    endfunction
33
34    task automatic generate_stimulus(ref stim_t arr[]);
35        for (int i = 0; i < arr.size(); i++) begin
36            arr[i].data_in = $urandom_range(0, 255);
37            arr[i].op_sel = $urandom_range(0, 3);
38        end

```

```

39  endtask
40
41  task automatic drive_stim();
42      for (int i = 0; i < stim_arr.size(); i++) begin
43          data_in = stim_arr[i].data_in;
44          op_sel  = stim_arr[i].op_sel;
45          #1;
46          collect_output_data(data_out);
47      end
48  endtask
49
50  task automatic golden_model(ref bit [7:0] exp_aa[int]);
51      bit [7:0] exp_val;
52      for (int i = 0; i < stim_arr.size(); i++) begin
53          case (stim_arr[i].op_sel)
54              2'b00: exp_val = stim_arr[i].data_in + 1;
55              2'b01: exp_val = stim_arr[i].data_in - 1;
56              2'b10: exp_val = ~stim_arr[i].data_in;
57              2'b11: exp_val = stim_arr[i].data_in << 1;
58              default: exp_val = stim_arr[i].data_in;
59          endcase
60          exp_aa[i] = exp_val;
61      end
62  endtask
63
64  task automatic collect_output_data(input bit [7:0] dut_out);
65      actual_q.push_back(dut_out);
66  endtask
67
68  task automatic check_results(ref bit [7:0] exp_aa[int], ref bit [7:0] act_q[$]);
69      for (int i = 0; i < act_q.size(); i++) begin
70          num_checks++;
71          if (act_q[i] === exp_aa[i]) begin
72              $display("[CHECK][PASS] idx=%0d A=0x%0h op_sel=%0b exp=0x%0h act=0x%0h",
73                  i, stim_arr[i].data_in, stim_arr[i].op_sel, exp_aa[i], act_q[i]);
74          end else begin
75              num_errors++;
76              $display("[CHECK][FAIL] idx=%0d A=0x%0h op_sel=%0b exp=0x%0h act=0x%0h",
77                  i, stim_arr[i].data_in, stim_arr[i].op_sel, exp_aa[i], act_q[i]);
78          end
79      end
80  endtask
81
82  task automatic reconfigure_stim(ref stim_t arr[]);
83      arr.shuffle();
84  endtask
85
86  initial begin
87      num_checks = 0;
88      num_errors = 0;
89
90      $display("=== PASS 1: 100 stimulus items ===");
91      configure_stim_storage(100);
92      generate_stimulus(stim_arr);

```

```

93     drive_stim();
94     golden_model(expected_aa);
95     check_results(expected_aa, actual_q);
96
97     $display("=== PASS 2: reconfigure to 200 stimulus items ===");
98     configure_stim_storage(200);
99     generate_stimulus(stim_arr);
100    drive_stim();
101    golden_model(expected_aa);
102    check_results(expected_aa, actual_q);
103
104    $display("=== PASS 3: reorder 200 items ===");
105    reconfigure_stim(stim_arr);
106    actual_q.delete();
107    golden_model(expected_aa);
108    drive_stim();
109    check_results(expected_aa, actual_q);
110
111    $display("=== SUMMARY: checks=%0d errors=%0d ===", num_checks, num_errors);
112    $finish;
113 end
114 endmodule

```

3.3 Automation Script: run.do

Listing 3: QuestaSim automation script

```

1 vlib work
2 vlog -sv -work work multi_op_processor.sv tb_multi_op_processor.sv
3 vsim -c work.multi_op_processor_tb
4 add wave *
5 run -all

```

4 Verification & Results

The verification script completed and verified 500 checks in total (100 for Pass 1, 200 for Pass 2, and 200 for Pass 3, where Pass 3 verifies full correctness after array shuffling via `.shuffle()`). The simulation resulted in 0 errors.

4.1 Waveform Screenshots

The following two figures show the QuestaSim waveform views, displaying transitions of `data_in`, `op_sel`, and `data_out` across the three verification passes alongside the checker counters.

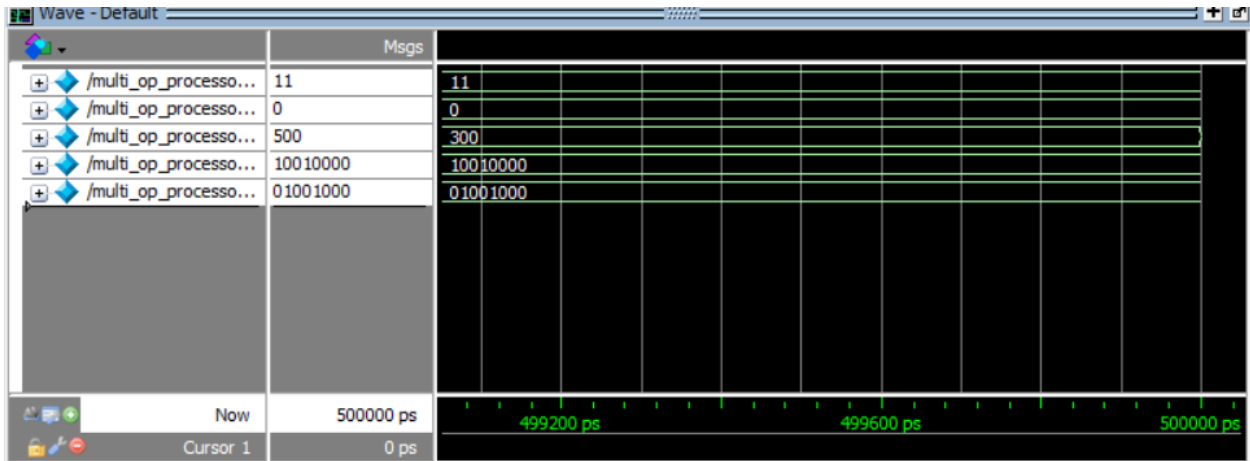


Figure 1: QuestaSim Waveform View — Full Overview

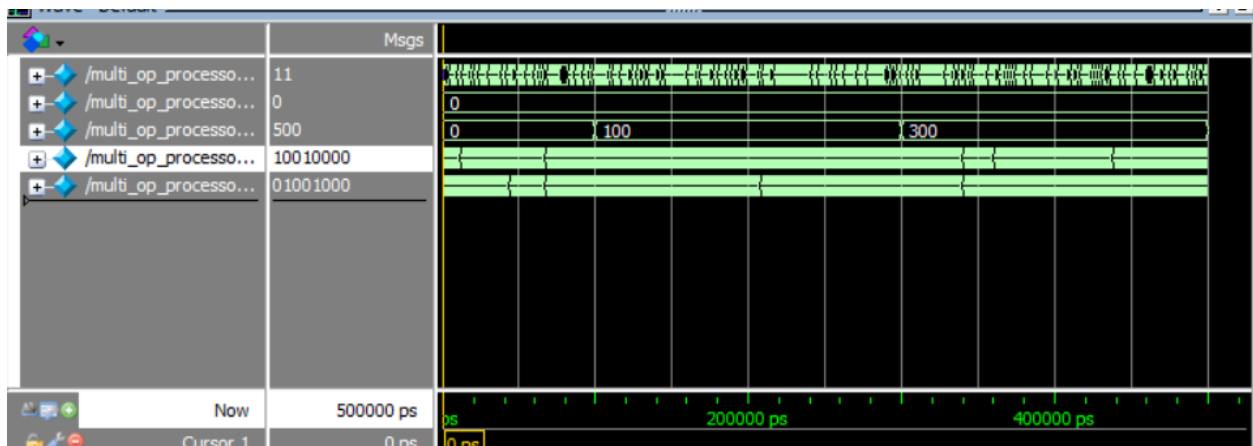


Figure 2: QuestaSim Waveform View — Zoomed-in Detail

4.2 Console Log Output

Below is a screenshot of the compilation and simulation transcript showing the execution flow and the final checker outcomes:

```

* [CHECK][PASS] idx=199 data_in=0x48 op_sel=11 expected=0x90 actual=0x90
* [CHECK] --- 200 checks this pass ---
* === SUMMARY: total_checks=500 total_errors=0 ===
* RESULT: ALL CHECKS PASSED
* ** Note: #finish      : E:/courses/Digital IC design Course/Digital Verification Course/system verilog assignments/assignment 2/tb_multi_op_processor.sv(133)
*      Time: 500 ns  Iteration: 0  Instance: /multi_op_processor_tb

```

Figure 3: ModelSim/QuestaSim Simulation Log Output